

DOCKER

INTRODUCTION

by

Marek Goldmann

10th Dec 2013



docker

DOCKER?

A project to create and manage containers.

CONTAINERS?

Yes, a lightweight operating system virtualization.

HOW THIS COMPARES TO A VIRTUAL MACHINE?

It's **completely** different.

COMPARISON

	Virtual Machine (KVM, VMware)	Container (LXC, OpenVZ)
Hardware	Simulated	Uses it (almost) directly
Supported OS'es	Almost any	Only Linux
Space	User space	Kernel space
Separation	Full	Control Groups (cgroups)
Startup time	Seconds to minutes	Miliseconds
Scalability	A few	Sky is the limit (thousands)
Custom kernel	Yes	No
Enterprise features (live migration, etc)	Yes	No
Ease of creation	Moderate	Easy
Time consumption of creation	High	Low
Size	HUGE	Small

WHO'S BEHIND DOCKER?

dotCloud* and a **powerful community**.

* dotCloud, Inc. was recently **renamed to Docker, Inc.**

SPEAKING OF COMMUNITY...

- **GitHub**
 - 501 watches
 - 6948 stars
 - 847 forks
 - 406 open issues
 - 69 pull requests
- Over **200 contributors**
 - avg 13 commits / day
 - **92% are external**
- Many meetups all over the world
 - **15** active **meetup** groups

Achieved after **10 months** since first commit to GitHub

GLOSSARY

- **Image:** a *package* with filesystem and data
- **Container:** a running *image*
- **Registry:** place where *images* are hosted/shared

MAIN CONCEPTS

- An image can be linked to a **parent** image
- Image not having a parent is called **base** image
- Container has a state, whereas image doesn't
- You can launch multiple containers from an image

DOCKER COMPONENTS

MANAGER

Running, stopping, attaching, etc... containers

BUILDER

Creating images

IMAGE REGISTRY

Storage for images

CREATING DOCKER IMAGES

It's simple! Use **Dockerfiles**.

DOCKERFILE

A **plain text** file with instructions that **automate** building the image.

DOCKERFILE EXAMPLE

```
# Base on the Fedora 20 image
FROM mattdm/fedora

# Upgrade the image
RUN yum -y update

# Install java
RUN yum -y install java-1.7.0-openjdk-devel wget

# Get WildFly
RUN wget http://download.jboss.org/wildfly/8.0.0.Beta1/wildfly-8.0.0.Beta1.tar.gz -O /
RUN tar -xf /opt/wildfly.tar.gz -C /opt

# Run WildFly after container boot
ENTRYPOINT /opt/wildfly-8.0.0.Beta1/bin/standalone.sh -c standalone-ha.xml -b 0.0.0.0
```

This should be saved as Dockerfile

BASIC INSTRUCTIONS

- **FROM:** the image we're basing on / extending
- **RUN:** The command to execute at the build time
- **ENTRYPOINT:** The command to be executed after the container boot

But there is **a lot more!**

BUILDING THE IMAGE

`docker build .`

BUILDING THE IMAGE

```
$ docker build .  
Uploading context 10240 bytes  
Step 1 : FROM mattdm/fedora  
  
[SNIP]  
  
Step 4 : RUN wget -q http://download.jboss.org/wildfly/8.0.0.Beta1/wildfly-8.0.0.Beta1  
----> Running in 41467b97b423  
----> 112766c23202  
Step 5 : RUN tar -xf /opt/wildfly.tar.gz -C /opt  
----> Running in d07d0c5c3dab  
----> c53f9f4ceb1e  
Step 6 : ENTRYPOINT /opt/wildfly-8.0.0.Beta1/bin/standalone.sh -c standalone-ha.xml -b  
----> Running in a9b2cecf4db5  
----> 5894688e4f2e  
Successfully built 5894688e4f2e
```

We've built image 5894688e4f2e

MODIFYING THE IMAGE

1. Change the Dockerfile
2. Execute the `build` command once again

LET'S TALK ABOUT SPEED

```
$ time docker build .  
Uploading context 10240 bytes  
Step 1 : FROM mattdm/fedora  
----> 0f3e92b4e94d  
  
[SNIP]  
  
Step 4 : RUN wget -q http://download.jboss.org/wildfly/8.0.0.Beta1/wildfly-8.0.0.Beta1  
----> Using cache  
----> 2ad6655478dd  
Step 5 : RUN tar -xf /opt/wildfly.tar.gz -C /opt  
----> Using cache  
----> c0f63f06e9ec  
Step 6 : RUN touch /etc/something  
----> Running in 8426dd5570af  
----> 7ef016a00cde  
Step 7 : ENTRYPOINT /opt/wildfly-8.0.0.Beta1/bin/standalone.sh -c standalone-ha.xml -b
```

We've built **a new** image e757e0ea05d7

Yes, **300ms!**

HOW DOES IT WORK

1. Prepare to execute an instruction
 - Was the instruction executed before?
 - **yes** - use the **previous result**, go to #3
 - **no** - execute the instruction, go to #2
2. Was the instruction executed without error?
 - **yes - commit**, go to #3
 - **no** - fail the build
3. Is this the end of instructions?
 - **yes - commit**, finish!
 - **no** - go to #1

LET'S TALK ABOUT SIZE

```
$ docker images
```

REPOSITORY	TAG	ID	CREATED	SIZE
<none>	<none>	e757e0ea05d7	37 minutes ago	12.29
<none>	<none>	5894688e4f2e	46 minutes ago	12.29

Yes, **12.29kB!**

IMAGE REPOSITORIES

1. **Private:** for example hosted inside a company
2. **Public:** index.docker.io

DOCKER CLI COMMANDS

DOCKER PULL

Pull image or repository to the registry

DOCKER PUSH

Push image from registry to a repository

DOCKER RUN

Run a command in a **new container** based on the specified image

DOCKER STOP

Stop a container

DOCKER START

Start a stopped container

DOCKER RM

Remove a stopped container

DOCKER RMI

Remove an image from registry

OK, GREAT, **BUT...**

USE-CASES?

- Want your own **PaaS**?
- Automated testing and **CI**
- Deploying **scalable** apps
- Application **delivery platform**

PAAS

It's (fairly) easy to create a PaaS with Docker
There is even a **tutorial** to create one in 5 minutes...

OPENSIFT

<https://github.com/openshift/openshift-pep/blob/master/openshift-pep-010-docker-cartridges.md>

CONTINUOUS INTEGRATION

1. You commit
2. CI builds the app
3. CI creates an image with the new app
4. CI starts a container from new image
5. Integration tests are executed on a **real** application
6. **Profit!**

SCALABLE APPS

- **Requirement:** prepare your app to be easily scalable
- Starting a container is fast and cheap
- Extending a (for example WildFly) cluster is easy
- **Profit!**

DELIVERY PLATFORM

Build once - run anywhere!

- Downloading is easy and fast
- Upgrading images is easy too

PROFIT!

WHERE TO START?

```
yum install docker-io
```

[**http://docs.docker.io/en/latest/use/basics/**](http://docs.docker.io/en/latest/use/basics/)

THANKS!



Attributions:

<https://www.docker.io/>

